

MARIO: A Superadditive Multi-Algorithm Interworking Optimization Framework for Analog Circuit Sizing

Wangzhen Li¹, Yuan Meng¹, Ruiyu Lyu¹, Changhao Yan¹, Keren Zhu¹, Zhaori Bi^{1,*}, Dian Zhou^{1,2,†}, Xuan Zeng^{1,*}

¹ State Key Lab of ASIC & System, Microelectronics Department, Fudan University, Shanghai, China

² Department of Electrical Engineering, the University of Texas at Dallas, Richardson, Texas, USA

Abstract—Numeric optimization methods are widely utilized to tackle complex analog circuit sizing problems, where the challenges include expensive simulations, non-linearity, and high parameter dimensionality. However, the diverse characteristics exhibited by different circuits result in varied optimization landscapes, making it difficult to identify a single algorithm that consistently outperforms others across all problems. In this paper, we introduce a multi-algorithm interworking optimization framework, which achieves optimization superadditivity based on a pool of member algorithms and a powerful algorithm-interworking protocol. We propose a computing resource reallocation method, which employs multi-task Gaussian process regression and portfolio optimization techniques, leading to flexible and prudent online adaption of member algorithms. To efficiently utilize the computing resources for local exploitation, an evaluation data broadcast strategy enables cooperativeness across member algorithms. Besides, algorithms with different modeling overheads are integrated time-adaptively via an asynchronous parallelization mechanism. Comparative experiments against state-of-the-art algorithm-combining tools and optimization algorithms demonstrate the superiority of the proposed optimization framework.

Index Terms—Analog circuit sizing, optimization framework, superadditivity, black-box optimization algorithm, algorithm-combining tool.

I. INTRODUCTION

Analog integrated circuit automation remains in high demand to expedite the time-to-market delivery of industrial products, and the sizing stage is a bottleneck for human practitioners since both subtle expert experience and labor-intensive repetitive work are required. Therefore, various optimization algorithms [1]–[10] have been proposed to automate the analog circuit sizing, dealing with the challenges of **expensive simulations**, **non-linearity**, and **high parameter dimensionality**. However, while the diverse characteristics of analog circuits, including **topologies**, **specifications**, and **technologies**, result in varied optimization landscapes, existing works merely propose algorithms based on a limited number of case-study analog circuit instances, incurring poor optimization generalizability.

In this work, we recognize that it is difficult to identify or design a single algorithm that consistently outperforms others across diverse analog circuits. This difficulty is fatally incurred by the famous No Free Lunch (NFL) theorem [11], which indicates that the superiority of an optimization algorithm on several problems is always accompanied by inferiority on several other problems. Thus, given a pool of candidate algorithms, it is imperative to develop a universal optimization framework for sizing diverse analog circuits.

We introduce a critical concept of **superadditivity** as the destination of optimization frameworks, inspired by the active research in academic fields and landmark success in modern technologies, e.g. Gestalt psychology [12], coalition games [13], compute unified device architecture (CUDA) [14] and large language models (LLMs) [15]. Superadditivity means that the performance of a whole system should be qualitatively better than the sum of the performance of all parts,

e.g. CUDA and LLMs respectively organize graphics processing kernels and self-attention modules as their parts, and then emerge with fundamentally superior computing capacity as systems. In the proposed optimization framework, we take member algorithms as parts and pursue the optimization superadditivity, which depends on two prerequisites as follows.

- ① Construct a heterogeneous pool of optimization algorithms.
- ② Develop an efficient and effective algorithm-interworking protocol.

For prerequisite ①, we extensively analyze and collect different algorithms, focusing on the optimization characteristics in terms of **sampling efficiency**, **convergency**, and **scalability**. Additionally, incorporating expensive simulations in the optimization loop, the ability of **batch sampling** is important for a candidate algorithm to speed up the optimization progression. In our framework, we integrate evolutionary algorithms (EAs), Bayesian optimization (BO), and mathematical derivative-free optimization (DFO), of which the complementary strengths and shortages are summarized in Table I.

TABLE I
THE STRENGTHS AND SHORTAGES OF DIFFERENT ALGORITHMS

Algorithm	Sampling Efficiency	Convergency	Scalability
EAs [1]–[3], [16]	Low	Low	High
BO [7]–[10], [17]	High	Medium	Low
DFO [18], [19]	Medium	High	Medium

EAs, such as differential evolution (DE) [1], genetic algorithm (GA) [2], and particle swarm optimization (PSO) [3], are compatible with batch sampling and scalable for high-dimensional problems, since these algorithms evolve an elite population of visited design points for candidate suggestions, and the related heuristical operators are computing-inexpensive. However, EAs lack strategies to capture fine-grained features of the optimization landscapes and hence have weaknesses in local exploitation. **BO** balance the tradeoff between exploration and exploitation by optimizing acquisition functions in advance of candidate suggestions and thus achieving high sampling efficiency. The formulation of acquisition functions are based on online regression models. However, given high-dimensional design variables, online regression modeling is prone to underfit, incurring over-exploration in BO. Therefore, state-of-the-art BO algorithms, e.g. TuRBO [8], cVTS [9] utilize localization techniques to improve the scalability. As for **DFO**, Nelder-Simplex [18] and BOBYQA [19] are influential, and the latter is popular nowadays due to good sampling efficiency. BOBYQA also employs online models but differs from BO due to the focus on local convergence rather than global search, thereby suggesting candidate points by solving mathematical subproblems rather than optimizing acquisition functions. Since DFO is generally local and serial-limited, we can run several optimization procedures in parallel with different starting points to enable global exploration and batch sampling. This multi-starting point (MSP) strategy is widely adopted [20], although it is still limited by the random selection of starting points, which is inefficient for global exploration in heavily multi-modal optimization landscapes.

*Corresponding authors: {zhaori_bi, xzeng}@fudan.edu.cn

†Emeritus professor, the University of Texas at Dallas.

This research is supported partly by National Key R&D Program of China 2020YFA0711900, 2020YFA0711901, partly by National Natural Science Foundation of China (NSFC) research projects 92473206, 62304052, 62474050, 92473207, 62474051.

For prerequisite ②, algorithm-combining protocols, including passive algorithm selection, active algorithm selection, chaining, and splitting, have been actively studied by recent works. **Passive algorithm selection** [21], [22] picks one algorithm from a pre-built decision tree at the beginning of the optimization run, according to the given dimensionality of parameters, budget of evaluations, amount of computing resources, etc. **Active algorithm selection** [23]–[25] means starting multiple optimization algorithms in parallel and selecting one of them to continue with a decision-maker after a certain number of evaluations. The decision-maker can be realized either with heuristical strategies or pre-trained neural networks. **Chaining** [26] involves running several algorithms sequentially, with the following ones initialized using the evaluation information from previous ones. **Splitting** [27] partitions the design variables, and then each group of them is optimized by its own optimizer. Further, active algorithm selection, chaining, and splitting can be integrated into **hybrid passive algorithm selection**. It has been implemented by Nevergrad [27], making the related ABBO [28] and Shiwa [29] the state-of-the-art algorithm-combining tools.

The main drawback of existing algorithm-combining protocols is the reliance on massive prior experimental experience, which is a burden for the developers and the combined optimization performance is prone to deteriorate in tackling previously unseen problems. Therefore, considering extensibility and generalizability, active algorithm selection with a heuristical decision-maker is more **flexible**. Nevertheless, current heuristical active decision-makers e.g. choosing one algorithm once based on the best observation [23], take speculative risks to produce satisfactory optimization results. Targeting a more **prudent** algorithm selection framework, it is necessary to refine the switching process across member algorithms and employ online modeling techniques to make the adaption of algorithms more comprehensive. In addition, it is also crucial to make the member algorithms **cooperative**, which means enabling heterogeneous algorithms to utilize the evaluation data from each other, hence efficient strategies for evaluation information extraction and transfer are dispensable. Besides, for analog circuits with simulation time limited to several seconds, BO and DFO algorithms suffer from non-neglectable modeling overheads and get distinctly slower than evolutionary algorithms. Therefore, a powerful algorithm-interworking protocol should be **time-adaptive** across different algorithms.

To address the aforementioned challenges of flexibility, prudence, cooperativeness, and time-adaptivity for achieving optimization superadditivity, we propose an innovative multi-algorithm interworking optimization framework MARIO. Overall, we extensively collect a pool of heterogeneous optimization algorithms, run them in parallel, and develop a powerful algorithm-interworking protocol for joint optimization. Our main contributions are summarized as follows.

- 1) We propose an active computing resource reallocation method, achieving **flexible** and **prudent** online adaption of member algorithms. We employ a customized multi-task Gaussian process to model the optimization potential of member algorithms and utilize portfolio optimization to reallocate the computing resources, improving the optimization desirability periodically.
- 2) We propose an efficient evaluation data broadcast strategy, making the member algorithms **cooperative** for joint optimization. With the accumulation of evaluated design points, we incrementally partition the design space into Voronoi nests and filter out informative data to broadcast. This strategy is especially beneficial to local algorithms since they can be effectively reinitialized to exploit promising design regions.
- 3) We propose a practical asynchronous parallelization mech-

anism, leading to **time-adaptive** algorithm integration in MARIO. If some algorithms exhibit non-negligible modeling overheads, this mechanism adapts faster algorithms to evaluate more design points and get more competitive to gain computing resources, thus assuring reasonable optimization performance.

With fair utilization of computing resources, MARIO achieves more than $3.83\times$ optimization speedup on a test-function benchmark suite and more than $2.59\times$ optimization speedup on an analog-circuit benchmark suite against state-of-the-art black-box optimization algorithms and algorithm-combining tools, based on the overall optimization performance measured by data profiles (see Sect. II-B). Python implementation and experimental results of MARIO can be found at <https://github.com/RogerLee0038/MARIO>.

II. BACKGROUND

A. Problem Formulation

The automatic sizing for a given analog circuit can be formulated as an optimization problem p as 1.

$$\begin{aligned} p : \quad & \arg \min_{\mathbf{x} \in \mathbb{R}^{d_p}} f_p(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{l}_p \leq \mathbf{x} \leq \mathbf{u}_p, \end{aligned} \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^{d_p}$ represents the d_p -dimensional vector of design variables, f_p represents the objective function, and $\mathbf{l}_p \sim \mathbf{u}_p$ represent the box constraints of the design space, which are defined by the experience of circuit designers and the rules of process design kits.

Considering constraints on circuit specifications, the objective function is formulated as a weighted sum as $f_p(\mathbf{x}) = s_{p,0}(\mathbf{x}) + \lambda_{p,1}c_{p,1}(\mathbf{x}) + \dots + \lambda_{p,n_s}c_{p,n_s}(\mathbf{x})$, where $s_{p,0}$ is the target specification to be minimized, $c_{p,1:n_s} = \max(0, s_{p,1:n_s})$ are constraint violation of other specifications, and $\lambda_{p,1:n_s}$ are weight multipliers.

B. Performance Measurement

Given a set $\mathcal{P}$ of diverse analog circuit sizing problems and a set $\mathcal{M}$ of optimizers, we measure the overall performance of each optimizer $m \in \mathcal{M}$ by the data profile [30] as 2.

$$D_m(a) = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} T_{p,m}(a), \quad (2)$$

where the input variable a represents the number of objective evaluations or the optimization time. $T_{p,m}(a)$ measures the optimization performance of m on a single problem p as 3.

$$T_{p,m}(a) = \frac{f_p(\mathbf{x}_0^m) - f_p(\mathbf{x}_a^m)}{f_p(\mathbf{x}_0^m) - f_p^L}, \quad (3)$$

where $\mathbf{x}_0^m$ is the optimization starting point of m , $\mathbf{x}_a^m$ is the a -th point suggested by m , and f_p^L is the smallest objective value obtained by any $m \in \mathcal{M}$ within a given optimization budget $\hat{\alpha}_p$. This formulation of $T_{p,m}(a)$ measures whether the problem p can be well optimized by m , based on the objective value reduction $f_p(\mathbf{x}_0) - f_p(\mathbf{x}_a)$ relative to the best possible reduction $f_p(\mathbf{x}_0) - f_p^L$.

In addition, considering different problem dimensionality d_p , the optimization budget $\hat{\alpha}_p$ is usually normalized as $\hat{\kappa} \cdot (d_p + 1)$, where $\hat{\kappa}$ represents a budget factor that is common for all $p \in \mathcal{P}$. Then, $D_m(a)$ can be normalized as $D_m(\kappa)$, with $\kappa = \alpha/(d_p + 1) \in [0, \hat{\kappa}]$.

C. Portfolio Optimization

For prudent reallocation of computing resources across member algorithms in MARIO, we treat the parallel workers for circuit simulations as funds and different optimization algorithms as investments, then utilize portfolio optimization [31] to periodically make

reallocation decisions. Specifically, given a collection of possible investments with a vector $\mathbf{r}$ of expected returns and a matrix $\mathbf{G}$ of covariances, an investor can construct a portfolio by putting a fraction ω_i of the available funds into investment i . Then, the desirability of the portfolio is measured and optimized as 4.

$$\begin{aligned} \arg \max_{\omega} \quad & \omega^T \mathbf{r} - \eta \omega^T \mathbf{G} \omega \\ \text{s.t.} \quad & \sum_{i=1}^N \omega_i = 1, \omega \geq 0, \end{aligned} \quad (4)$$

where η is a risk aversion parameter that can be small for conservative investors and large for more daring investors.

D. Multi-Task Gaussian Process

The return vector $\mathbf{r}$ and covariance matrix $\mathbf{G}$ in portfolio optimization are constructed as introduced in Sect. III-A, with the aid of a multi-task Gaussian process (MTGP) [32]. Given a d -dimensional input space $\mathbb{R}^d$ and M tasks, an MTGP builds a regression model based on an observation dataset $\mathcal{D} = \{(\boldsymbol{\kappa}_i, \mathbf{y}_i)\}_{i=1}^N$, where $\boldsymbol{\kappa}_{1:N} \in \mathbb{R}^d$ are training features and $\mathbf{y}_{1:N} \in \mathbb{R}^M$ are response labels. We adopt $\boldsymbol{\kappa}$ instead of $\mathbf{x}$ here because the MTGP in MARIO will take the normalized number of evaluations rather than the design variables of analog circuits as the training features. Specifically, an MTGP models the covariances over input features and the similarities over tasks with a kernel function $k^\kappa(\boldsymbol{\kappa}_i, \boldsymbol{\kappa}_j)$ and a positive semi-definite matrix $\mathbf{K}^{tas}$, respectively. After training, the MTGP predicts an arbitrary unobserved response at the input location $\boldsymbol{\kappa}^*$ of the task l with the posterior mean and variance as 5.

$$\mu_l(\boldsymbol{\kappa}^*) = (\mathbf{k}^\kappa(\boldsymbol{\kappa}^*) \otimes \mathbf{K}_l^{tas})^T (\mathbf{K}^\kappa \otimes \mathbf{K}^{tas} + \mathbf{I}_N \otimes \boldsymbol{\Lambda})^{-1} \mathbf{y} \quad (5a)$$

$$\sigma_l^2(\boldsymbol{\kappa}^*) = \mathbf{k}^\kappa(\boldsymbol{\kappa}^*, \boldsymbol{\kappa}^*) \cdot \mathbf{K}_{l,l}^{tas} - (\mathbf{k}^\kappa(\boldsymbol{\kappa}^*) \otimes \mathbf{K}_l^{tas})^T \cdot$$

$$(\mathbf{K}^\kappa \otimes \mathbf{K}^{tas} + \mathbf{I}_N \otimes \boldsymbol{\Lambda})^{-1} (\mathbf{k}^\kappa(\boldsymbol{\kappa}^*) \otimes \mathbf{K}_l^{tas}), \quad (5b)$$

where $\mathbf{y}$ is a vector containing the MN response values in the dataset ordered as $(\mathbf{y}_{11}, \dots, \mathbf{y}_{1M}, \dots, \mathbf{y}_{N1}, \dots, \mathbf{y}_{NM})$. $\otimes$ denotes the Kronecker product, $\mathbf{K}_{l,l}^{tas}$ and $\mathbf{K}_{l,l}^{tas}$ select the l -th row and the (l, l) element from $\mathbf{K}^{tas}$ respectively, $\mathbf{K}^\kappa$ represents the matrix of covariances between all pairs of $\boldsymbol{\kappa}_{1:N}$, $\mathbf{k}^\kappa(\boldsymbol{\kappa}^*)$ is a vector of covariances between $\boldsymbol{\kappa}^*$ and $\boldsymbol{\kappa}_{1:N}$, and $\boldsymbol{\Lambda}$ is an $M \times M$ matrix with the (l, l) element as σ_l^2 , i.e. the noise variance for the l -th task.

The hyperparameters in a MTGP include the parameters $\boldsymbol{\theta}_\kappa$ in the kernel function k^κ , the entries of matrix $\mathbf{K}^{tas}$ and the noise variances $\sigma_{1:M}^2$ in $\boldsymbol{\Lambda}$. They can be trained by maximizing the marginal likelihood according to the distribution $\mathbf{y}|\boldsymbol{\kappa}_{1:N} \sim \mathcal{N}(\mathbf{0}, (\mathbf{K}^\kappa \otimes \mathbf{K}^{tas} + \mathbf{I}_N \otimes \boldsymbol{\Lambda}))$, which is updated along with the dataset $\mathcal{D}$.

III. PROPOSED METHODS

In this section, we introduce the key techniques of MARIO in detail, including active computing resource reallocation, evaluation data broadcast, and asynchronous parallelization mechanism. The overall workflow of MARIO is depicted in Fig. 1.

A. Active Computing Resource Reallocation

Since the circuit simulations are expensive, we measure the computing resources involved in MARIO by the number of parallel simulation workers. These workers parallelize the simulations of the candidate design points, which are suggested batch by batch from member algorithms. Then, the active computing resource reallocation is implemented via a customized MTGP and portfolio optimization.

We employ an MTGP enhanced by exponentially decaying kernels to periodically extrapolate the optimization curves and quantify the optimization potential of member algorithms. Specifically, the current-best-curve progression of member algorithms is regarded as

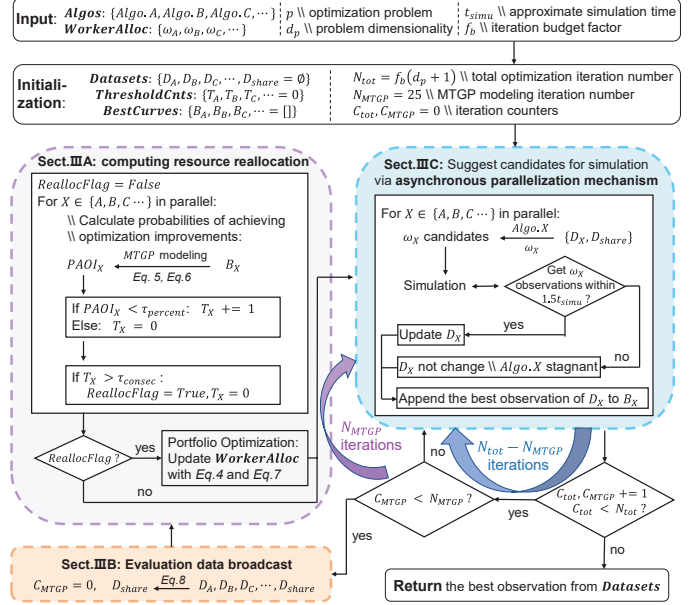


Fig. 1. The overall workflow of MARIO. Given a pool of *Algos*, MARIO runs them in parallel for N_{tot} iterations, while the evaluation data broadcast and the computing resource reallocation are conducted in N_{MTGP} iterations. For broadcast, MARIO maintains a dataset D_{share} , which contributes to the candidate suggestions of all member algorithms. For reallocation, the probability of achieving optimization improvements (PAOI) is calculated for member algorithms by modeling the *BestCurves*. If an algorithm exhibits a poor (less than τ_{percent}) PAOI for a consecutive number τ_{consec} of modeling rounds, the *WorkerAlloc* is updated by portfolio optimization.

different modeling tasks, with the normalized number of evaluations as the common input variable and the objective values as the responses. After modeling, the MTGP will predict the extrapolated objective values of different member algorithms through a posterior normal distribution. Based on the posterior mean μ and variance σ , the probability of achieving optimization improvement (PAOI) of each member algorithm is calculated as $\text{PAOI}(\boldsymbol{\kappa}^*; \mathcal{D}) = \Phi\left(\frac{v - \mu(\boldsymbol{\kappa}^*; \mathcal{D})}{\sigma(\boldsymbol{\kappa}^*; \mathcal{D})}\right)$, where v represents the current-best observed objective value, $\boldsymbol{\kappa}^*$ is a prediction point and Φ is the standard normal cumulative distribution. Note that in addition to μ and σ , we also obtain a matrix $\mathbf{C}$ of extrapolated objective covariances across member algorithms, which will be used later in 7 for portfolio optimization.

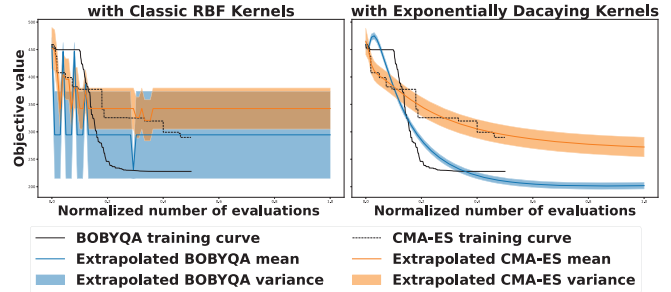


Fig. 2. Extrapolation performance with two different kernels for two different algorithms while optimizing a 10D Ackley test function.

The ability to extrapolate optimization curves is achieved with exponentially decaying kernels [33] as 6.

$$k(\boldsymbol{\kappa}, \boldsymbol{\kappa}') = \int_0^\infty e^{-\xi(\boldsymbol{\kappa} + \boldsymbol{\kappa}')} \frac{\beta^\alpha}{\Gamma(\alpha)} \xi^{\alpha-1} e^{-\xi\beta} d\xi = \frac{\beta^\alpha}{(\boldsymbol{\kappa} + \boldsymbol{\kappa}' + \beta)^\alpha}, \quad (6)$$

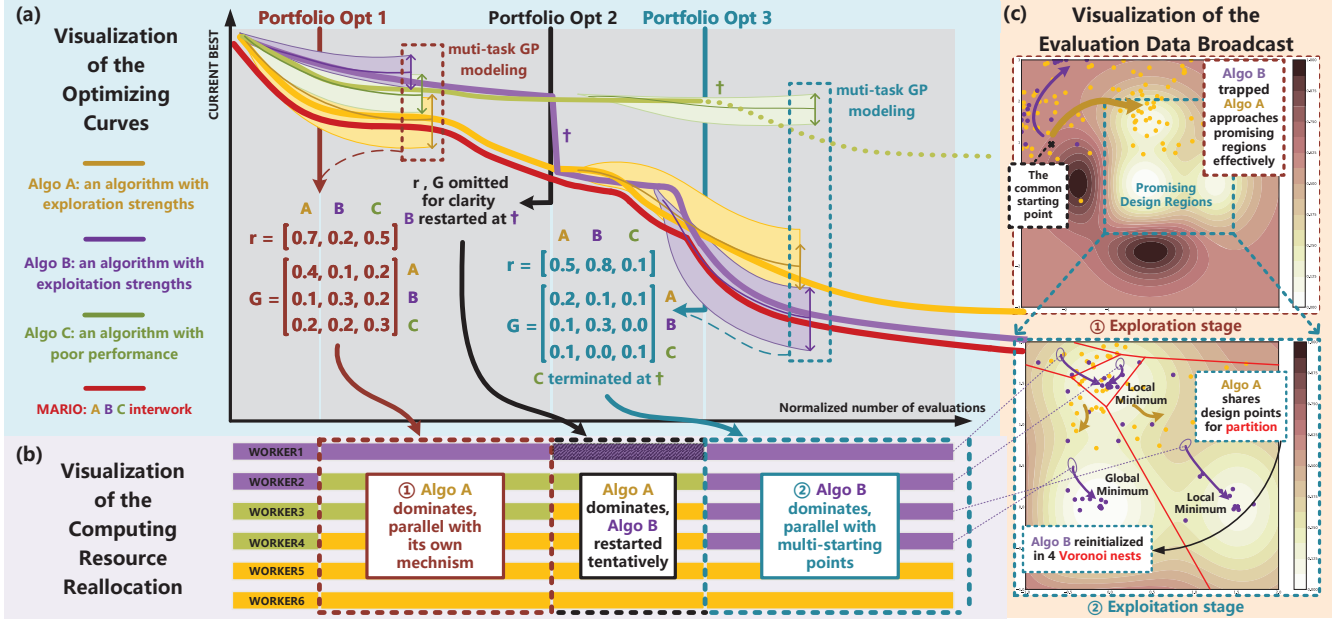


Fig. 3. An illustration of the optimization progression of MARIO. (a) A visualization of three optimizing curves and three periods of computing resource reallocation. The red curve results from combining the three heterogeneous member algorithms A, B, and C within MARIO. The $\mathbf{r}$ and $\mathbf{G}$ are modeled by the multi-task Gaussian process and the parallel workers are reallocated by portfolio optimization. (b) A visualization of the evolution of parallel computing resources, where algorithm A dominates optimization after the first two periods of portfolio optimization, and algorithm B dominates in turn after the third period of portfolio optimization. (c) A visualization of the evaluation data broadcast strategy. In the early exploration stage, algorithm B is trapped in a local minimum, while algorithm A can approach the promising design regions. In the later exploitation stage, the design space is partitioned into Voronoi nests based on the shared evaluated design points. Then algorithm B is enabled to reinitialize its optimization procedures, exploiting 4 promising regions in parallel.

which is constructed by integrating an infinite set of exponentially decaying functions of the form $f(\kappa, \xi) = e^{-\xi\kappa}$, with κ representing the 1-dimensional input variable. $\frac{\beta^\alpha}{\Gamma(\alpha)} \xi^{\alpha-1} e^{-\xi\beta}$ is a non-negative mixing measure weighting the integration range with parameters $\alpha, \beta > 0$, and $\Gamma(\cdot)$ is the gamma function. As shown in Fig. 2, exponentially decaying kernels produce more informative and smoother extrapolation performance than classic RBF kernels. Additionally, since the MTGP is based on the 1-dimensional input variable and is efficiently accelerated in MARIO through black-box matrix-matrix multiplication techniques [34], its computational cost is neglectable in comparison to the overhead of circuit simulations.

If a member algorithm exhibits a poor POAI for a certain consecutive number of MTGP-modeling periods, the computing resource reallocation is activated, based on the portfolio optimization as formulated in 4, where the $\mathbf{r}$ and $\mathbf{G}$ are constructed as 7.

$$\mathbf{r} = \max(\mathbf{0}, \boldsymbol{\mu} - \mathbf{v}) \circ \boldsymbol{\omega}_{cur}^{(-1)} \quad (7a)$$

$$\mathbf{G} = \mathbf{C} \circ (\boldsymbol{\omega}_{cur}^T \boldsymbol{\omega}_{cur})^{(-\frac{1}{2})}, \quad (7b)$$

where $\boldsymbol{\mu}$ represents the vector of extrapolated objective means, $\mathbf{C}$ represents the matrix of extrapolated objective covariances, $\boldsymbol{\omega}_{cur}$ represents the current allocation of computing resources, $\circ$ represents the Hadamard product and $\circ^{(-1)}$ represents the element-wise power.

We additionally introduce two constraints for portfolio optimization. The first constraint is $\sum_i |\boldsymbol{\omega}_i - \boldsymbol{\omega}_{cur}| \leq \Omega$, which restricts the total turnover Ω of computing resources, considering that the optimization potential of member algorithms is approximately in proportion to the computing resources within a relative-small linearized range. The second constraint is $\boldsymbol{\omega}_i \leq \boldsymbol{\omega}_{max}$ for local algorithms, which avoids the waste of computing resources due to meaningless local exploitation. $\boldsymbol{\omega}_{max}$ represents the number of promising design regions, which is derived systematically later in Sect.III-B,

Besides, we adopt a dynamic risk aversion parameter η for portfolio optimization in 4 to make the computing resource reallocation more comprehensive. In the early optimization stages, η is relatively large to encourage member algorithms with high optimization variance. It is reasonable since these algorithms generally serve for global exploration and need to gobble up computing resources to reach promising design regions quickly. Then η gets smaller during the optimization progression, gradually preferring local algorithms that can steadily exploit the promising design regions.

A visualization of three periods of computing resource reallocation among three member algorithms is presented in Fig. 3(a) and 3(b).

B. Evaluation Data Broadcast

To empower optimization superadditivity in MARIO, it is critical to make the member algorithms cooperate with each other. Therefore, we maintain a parallel broadcast process to periodically collect the evaluated design points from different member algorithms and extract informative evaluation data to transfer.

To assure the efficiency of evaluation data broadcast, it is necessary to prune uncompetitive observations because these redundant data heavily retard and even mislead the progression of optimization. Hence, with the accumulation of evaluated design points, We incrementally partition the design space into Voronoi nests and filter out information of promising design regions to broadcast.

The Voronoi nests are constructed by clustering the design points, and the clustering is based on the objective-weighted distance as 8.

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = w \sqrt{\frac{d_s^2}{d_p}} + (1 - w) \sqrt{d_o^2}, \quad (8)$$

where $d_s = \|\mathbf{x}_i - \mathbf{x}_j\|$ is the spatial distance, $d_o = \sqrt{(f_i - f_j)^2}$ is the objective distance, d_p is the dimensionality of the design

variables, and w is a weight parameter. Consequently, the design points that are spatially close and qualitatively similar will be clustered into the same Voronoi nest. Note that the total number of Voronoi nests can be controlled by limiting the maximum number of design points inside a nest [9]. After partition, we score each Voronoi nest according to the best-achieved objective value, and the involved design point that achieves this value is regarded as the representative design point of the nest. Based on the scores, we filter out competitive Voronoi nests, which exactly represent the promising design regions.

While global algorithms mainly utilize representative design points to promote exploration, local algorithms can further make use of the information of promising design regions to reinitialize themselves, which leads to adaptive and effective local exploitation, significantly contributing to the superadditivity of MARIO. The reinitialization requires no extra sampling overheads since the broadcasted design observations can be directly reused. Moreover, considering that the serial-limited local algorithms are parallelized in MARIO through the MSP strategy, the computing resource allocation for these algorithms can be restricted according to the number of promising design regions, which accounts for ω_{max} in Sect.III-A.

Fig. 3(c) presents a visualization of the evaluation data broadcast.

C. Asynchronous Parallelization Mechanism

Heterogeneous algorithms naturally exhibit different computational overhead for the candidate-point suggestions, and thus there will be optimization blocks in MARIO if the evaluated design points are synchronously collected. To make MARIO time-adaptive, we design an asynchronous parallelization mechanism, which is important in practice, especially for sizing the module-level analog circuits with simulation time limited to several seconds.

The proposed mechanism employs a timeout parameter t_{out} for the poll of parallelly evaluated design points across member algorithms. If no points are provided by a slow algorithm within t_{out} , this algorithm will be considered stagnant and other faster algorithms will become relatively more competitive in gaining computing resources. In MARIO, we set $t_{out} = 1.5t_{simu}$, where t_{simu} is the allowed simulation time for a given target circuit. This setting indicates that the collection of evaluated design points from a member algorithm will be postponed once this algorithm exhibits a modeling overhead 1.5 times more than the simulation overhead.

IV. EXPERIMENTAL RESULTS

In this section, we show the superiority of MARIO against existing optimizers. The baseline optimizers include state-of-the-art BO algorithms for analog circuit sizing (**TurBO** [8] and **cVTS** [9]), widely-used evolutionary and DFO algorithms (**CMA-ES** [16], **DE** [27], **TBPSA** [35] and **BOBYQA** [36]), and state-of-the-art algorithm-combining tools (**ABBO** [28] and **Shiwa** [29]).

For MARIO, we realize an instance as **MARIO-1**, combining **TurBO**, **cVTS**, **CMA-ES**, **DE**, and **BOBYQA** w.l.o.g. In **MARIO-1**, **TurBO**, **CMA-ES**, and **DE** are expected to act as global algorithms, while **cVTS** and **BOBYQA** are expected to act as local algorithms. Note that we remove the original global exploration strategy in **cVTS** because the evaluation data broadcast strategy in MARIO is a more informative alternative and the main advantage of VTS lies in the boundary-restricted local exploitation. In addition, we realize a counterpart of **MARIO-1** as **MARIO-1se**, where the computing resource reallocation is disabled for ablation studies.

For fair utilization of computing resources, the number of parallel workers for all optimizers is 20. Specifically, all optimizers suggest 20 design points for parallel simulations in each iteration, either based

on the MSP strategy (for serial-limited optimizers such as **cVTS** and **BOBYQA**) or batch sampling (for parallel-enabled optimizers). Besides, we unify the constraint handling for all optimizers via the weighted-sum objective function as introduced in Sect.II-A.

We conduct optimization experiments on two benchmark suites and run all optimizations 10 times to measure the variances of random fluctuations. The random seeds are provided in our code repository for reproducibility. All experiments are conducted on a Linux server with Intel Xeon 6230 CPUs and 128GB RAM.

A. Test-Function Experiments

The test-function benchmark suite includes 17 widely-used functions i.e. Ackley, CosineMixture, Exponential, Griewank, Levy-Montalvo1, LevyMontalvo2, Neumaier3, Paviani, Rastrigin, Rosenbrock, Salomon, Schwefel, Sinusoidal, Michalewicz, SineEnvelope, EggHolder, and Rana. All these functions have scalable dimensions, and we choose the dimensionality of $\{20, 40\}$ for demonstration.

Fig. 4 presents the data-profile accumulating curves of different optimizers on the test-function benchmark suite, with the x-axis representing the normalized number of evaluations. This normalization indicates that given a test function with dimensionality d_p , the maximum number of evaluations is limited to $20 \cdot 50 \cdot d_p$, where 20 is the number of parallel workers and 50 is a fixed budget factor. **MARIO-1** and **MARIO-1se** significantly outperform other baseline optimizers, while **MARIO-1** is steadily better than **MARIO-1se**. In comparison with the best baseline optimizer **TurBO**, **MARIO-1** achieves $3.83\times$ speedup for the normalized number of evaluations.

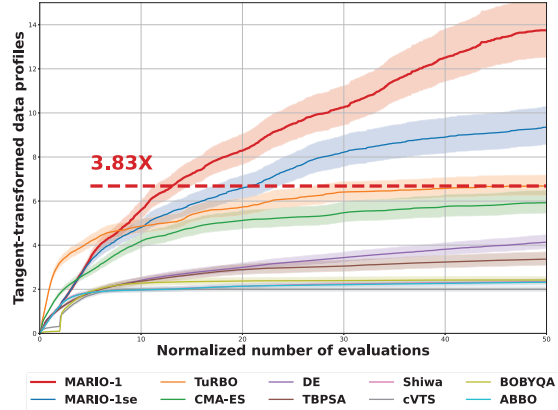


Fig. 4. The data-profile accumulating curves of different optimizers on the test-function benchmark suite. The translucent area shows a variance of 0.1σ .

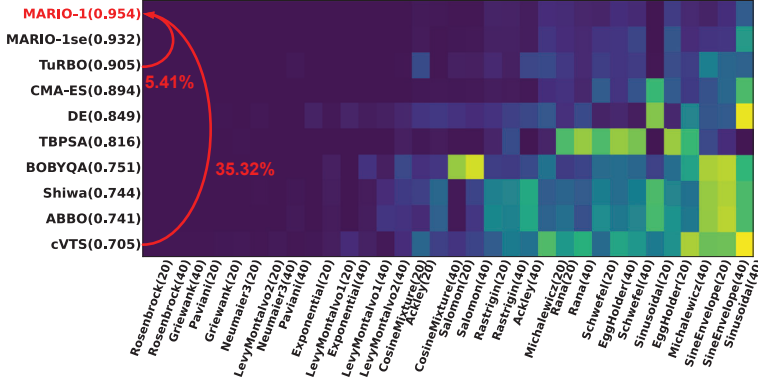
A breakdown of the best-achieved data profiles of different optimizers on the test-function benchmark suite is shown in Fig. 5(a), **MARIO-1** achieves competitive optimization performance across all test functions, leading to the highest data profile of 0.954, which is 5.41%~35.32% better than the data profiles of baseline optimizers.

B. Analog-Circuit Experiments

Table II summarizes the experiment setup for 15 analog circuits, with a part of them open-sourced [37] and available in our repository. The evaluation-number budget and optimization-time budget for analog circuits are $20 \cdot 20 \cdot (d_p + 1)$ and $t_{simu} \cdot 20 \cdot (d_p + 1)$ respectively, with the use of 20 parallel workers and a budget factor of 20.

Fig. 6 presents the data-profile accumulating curves of different optimizers on the analog-circuit benchmark suite, **MARIO-1** and **TurBO** significantly outperform other optimizers, while **MARIO-1** achieves $2.59\times$ and $1.91\times$ speedup compared to **TurBO** for the normalized number of evaluations and optimization time, respectively.

(a) The breakdown of data profiles on the test functions



(b) The breakdown of data profiles on the analog circuits

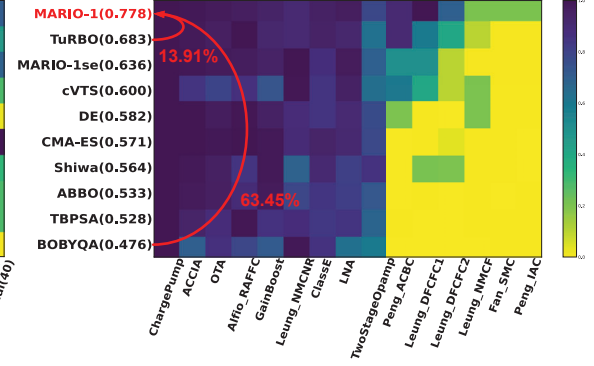


Fig. 5. The breakdown of the best-achieved data profiles for different optimizers on the test-function and analog-circuit benchmark suites. The x-axes represent different optimization problems p , the y-axis represents different optimizers m , and the grids visualize the values of $T_{p,m}$, with the darker color indicating better optimization performance. The row-wise average of $T_{p,m}$ is the data profiles, which are presented following each optimizer on two benchmark suites.

TABLE II
THE EXPERIMENT SETUP FOR ANALOG CIRCUITS

Name	d_p, t_{simu}	Specification Description	Evaluation Budget	Time Budget
Charge Pump	36, 10s	current deviation in 27 PVT corners	14800	2.06h
ACCIA	38, 10s	input referred noise, power, gbw, etc.	15600	2.17h
OTA	59, 5s	gain, ugf, etc.	24000	1.67h
ClassE	13, 10s	out power, dc power	5600	0.78h
LNA	19, 60s	NF, IIP3, etc.	8000	6.67h
GainBoost	20, 10s	gain, slew rate, etc.	8400	1.17h
TwoStageOpamp	10, 10s	loop gain, pm, etc.	4400	0.61h
8 circuits from AnalogGym [37]	23~33, 20s	10 specs, leading to FOM_{AMP}	9600~13600	2.67h~3.78h

In Fig. 6, no optimizer distinctly outperforms others in early exploration stages, thus the superiority of MARIO-1 mainly results from efficient local exploitation later. Evolutionary algorithms CMA-ES, DE, and TBPSA can not exploit promising design regions effectively, and neither can ABBO and Shiwa, since these algorithm-combining tools also select evolution algorithms in most cases. On the other hand, BOBYQA is talented in local exploitation but fails to reach promising design regions from the beginning. Besides, cVTS and BOBYQA are uncompetitive in our experiments because these two algorithms are serial-limited and merely utilize parallel computing resources with the MSP strategy, which is inefficient for exploring the heavily multi-modal optimization landscapes of analog circuits. The challenge of multi-modality also accounts for the poor performance of MARIO-1se, since it can't reallocate more computing resources for parallel local exploitation. Additionally, the modeling overheads of TuRBO and cVTS are non-neglectable, which is inferred by comparing the optimization performance along with time and the performance along with evaluations. The optimization efficiency of TuRBO is also limited by its restart strategy, which constructs new trust regions from scratch once the old ones are converged.

A breakdown of the best-achieved data profiles of different optimizers on the analog-circuit benchmark suite is shown in Fig. 5(b). The optimization superadditivity of MARIO-1 is remarkable, especially for difficult cases in the right part. The data profile of MARIO-1 is 13.91%~63.45% better than those of baseline optimizers.

V. CONCLUSIONS

In this work, we propose a superadditive multi-algorithm inter-working optimization framework MARIO for analog circuit sizing.

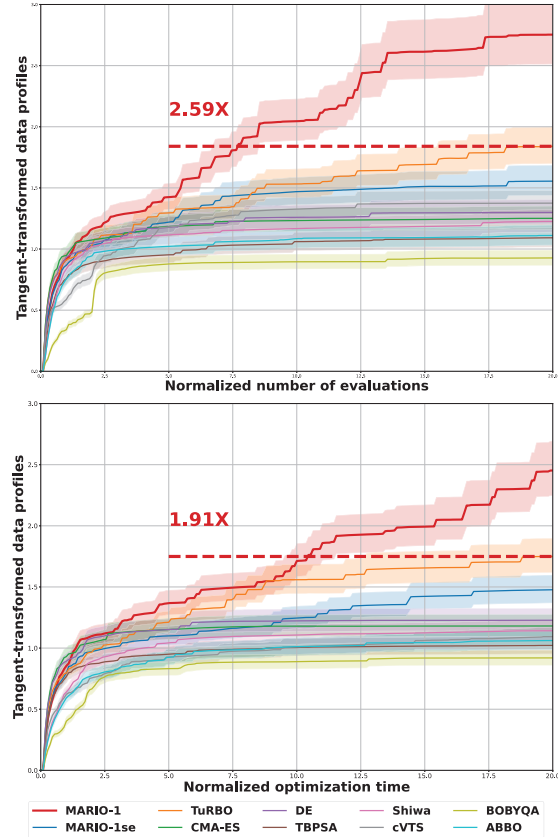


Fig. 6. The data-profile accumulating curves of different optimizers on the analog-circuit benchmark suite. The translucent area shows a variance of 0.1σ .

To demonstrate the superiority of MORIO, we implement MARIO-1, which combines 5 heterogeneous optimization algorithms. In comparison with the baseline optimizers for sizing 15 diverse analog circuits, MARIO-1 achieves more than $2.59\times$ evaluation-number speedup, and more than $1.91\times$ optimization-time speedup.

For future research, the application scenarios of MARIO are not necessarily limited to analog circuit sizing or single objective optimization, and we can further unleash the capacity of MARIO with multi-machine parallelism. Moreover, MARIO is extending-friendly for any new algorithms, with promising optimization superadditivity.

REFERENCES

- [1] B. Liu, Y. Wang, Z. Yu, L. Liu, M. Li, Z. Wang, J. Lu, and F. V. Fernández, "Analog circuit optimization system based on hybrid evolutionary algorithms," *Integration*, vol. 42, no. 2, pp. 137–148, 2009.
- [2] M. Taherzadeh-Sani, R. Lotfi, H. Zare-Hoseini, and O. Shoaie, "Design optimization of analog integrated circuits using simulation-based genetic algorithm," in *Signals, Circuits and Systems, 2003. SCS 2003. International Symposium on*, vol. 1, 2003, pp. 73–76 vol.1.
- [3] R. A. Vural and T. Yildirim, "Analog circuit sizing via swarm intelligence," *AEU-International journal of electronics and communications*, vol. 66, no. 9, pp. 732–740, 2012.
- [4] B. Liu, D. Zhao, P. Reynaert, and G. G. Gielen, "Gaspad: A general and efficient mm-wave integrated circuit synthesis method based on surrogate model assisted evolutionary algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 2, pp. 169–182, 2014.
- [5] A. F. Budak, M. Gandara, W. Shi, D. Z. Pan, N. Sun, and B. Liu, "An efficient analog circuit sizing method based on machine learning assisted global optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 5, pp. 1209–1221, 2021.
- [6] Y. Yang, H. Zhu, Z. Bi, C. Yan, D. Zhou, Y. Su, and X. Zeng, "Smartmsp: A self-adaptive multiple starting point optimization approach for analog circuit synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 3, pp. 531–544, 2017.
- [7] W. Lyu, P. Xue, F. Yang, C. Yan, Z. Hong, X. Zeng, and D. Zhou, "An efficient bayesian optimization approach for automated optimization of analog circuits," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 6, pp. 1954–1967, 2017.
- [8] K. Touloupas, N. Chouridis, and P. P. Sotiriadis, "Local bayesian optimization for analog circuit sizing," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1237–1242.
- [9] A. Zhao, X. Wang, Z. Lin, Z. Bi, X. Li, C. Yan, F. Yang, L. Shang, D. Zhou, and X. Zeng, "cvts: A constrained voronoi tree search method for high dimensional analog circuit synthesis," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [10] T. Gu, W. Li, A. Zhao, Z. Bi, X. Li, F. Yang, C. Yan, W. Hu, D. Zhou, T. Cui *et al.*, "Bbgp-sdfo: Batch bayesian and gaussian process enhanced subspace derivative free optimization for high-dimensional analog circuit synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [11] S. P. Adam, S.-A. N. Alexandropoulos, P. M. Pardalos, and M. N. Vrahatis, "No free lunch theorem: A review," *Approximation and optimization: Algorithms, complexity and applications*, pp. 57–82, 2019.
- [12] W. Köhler, "Gestalt psychology," *Psychologische forschung*, vol. 31, no. 1, pp. XVIII–XXX, 1967.
- [13] W. Saad, Z. Han, M. Debbah, A. Hjørungnes, and T. Basar, "Coalitional game theory for communication networks," *Ieee signal processing magazine*, vol. 26, no. 5, pp. 77–97, 2009.
- [14] J. Ghorpade, J. Parande, M. Kulkarni, and A. Bawaskar, "Gpgpu processing in cuda architecture," *arXiv preprint arXiv:1202.4347*, 2012.
- [15] Z. Wang, S. Mao, W. Wu, T. Ge, F. Wei, and H. Ji, "Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration," *arXiv preprint arXiv:2307.05300*, 2023.
- [16] N. Hansen, "The cma evolution strategy: A tutorial," *arXiv preprint arXiv:1604.00772*, 2016.
- [17] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, "Scalable global optimization via local bayesian optimization," *Advances in neural information processing systems*, vol. 32, 2019.
- [18] J. A. Nelder and R. Mead, "A simplex method for function minimization," *The computer journal*, vol. 7, no. 4, pp. 308–313, 1965.
- [19] M. J. Powell *et al.*, "The bobyqa algorithm for bound constrained optimization without derivatives," *Cambridge NA Report NA2009/06, University of Cambridge, Cambridge*, vol. 26, 2009.
- [20] L. Qian, Z. Bi, D. Zhou, and X. Zeng, "Automated technology migration methodology for mixed-signal circuit based on multistart optimization framework," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 11, pp. 2595–2605, 2014.
- [21] N. Baskiotis and M. Sebag, "C4. 5 competence map: a phase transition-inspired approach," in *Proceedings of the twenty-first international conference on Machine learning*, 2004, p. 10.
- [22] J. Rapin, P. Dorval, J. Pondard, N. Vasilache, M.-L. Cauwet, C. Couprie, and O. Teytaud, "Openly revisiting derivative-free optimization," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2019, pp. 267–268.
- [23] T. Weise, Z. Wu, and M. Wagner, "An improved generic bet-and-run strategy for speeding up stochastic local search," *arXiv preprint arXiv:1806.08984*, 2018.
- [24] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evolutionary computation*, vol. 27, no. 1, pp. 3–45, 2019.
- [25] P. Kerschke and H. Trautmann, "Automated algorithm selection on continuous black-box problems by combining exploratory landscape analysis and machine learning," *Evolutionary computation*, vol. 27, no. 1, pp. 99–127, 2019.
- [26] D. Molina, M. Lozano, and F. Herrera, "Memetic algorithm with local search chaining for continuous optimization problems: A scalability test," in *2009 Ninth International Conference on Intelligent Systems Design and Applications*. IEEE, 2009, pp. 1068–1073.
- [27] J. Rapin and O. Teytaud, "Nevergrad-a gradient-free optimization platform," 2018.
- [28] L. Meunier, H. Rakotoarison, P. K. Wong, B. Roziere, J. Rapin, O. Teytaud, A. Moreau, and C. Doerr, "Black-box optimization revisited: Improving algorithm selection wizards through massive benchmarking," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 3, pp. 490–500, 2021.
- [29] J. Liu, A. Moreau, M. Preuss, J. Rapin, B. Roziere, F. Teytaud, and O. Teytaud, "Versatile black-box optimization," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, 2020, pp. 620–628.
- [30] J. J. Moré and S. M. Wild, "Benchmarking derivative-free optimization algorithms," *SIAM Journal on Optimization*, vol. 20, no. 1, pp. 172–191, 2009.
- [31] F. J. Fabozzi, H. M. Markowitz, and F. Gupta, "Portfolio selection," *Handbook of finance*, vol. 2, 2008.
- [32] E. V. Bonilla, K. Chai, and C. Williams, "Multi-task gaussian process prediction," *Advances in neural information processing systems*, vol. 20, 2007.
- [33] K. Swersky, J. Snoek, and R. P. Adams, "Freeze-thaw bayesian optimization," *arXiv preprint arXiv:1406.3896*, 2014.
- [34] J. Gardner, G. Pleiss, K. Q. Weinberger, D. Bindel, and A. G. Wilson, "Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration," *Advances in neural information processing systems*, vol. 31, 2018.
- [35] M. Oquab, J. Rapin, O. Teytaud, and T. Cazenave, "Parallel noisy optimization in front of simulators: Optimism, pessimism, repetitions, population control," in *Workshop Data-driven Optimization and Applications at CEC*, 2019.
- [36] C. Cartis, L. Roberts, and O. Sheridan-Methven, "Escaping local minima with local derivative-free methods: a numerical investigation," *Optimization*, vol. 71, no. 8, pp. 2343–2373, 2022.
- [37] J. Li, H. Zhi, R. Lyu, W. Li, Z. Bi, K. Zhu, Y. Zeng, W. Shan, C. Yan, F. Yang *et al.*, "Analoggym: An open and practical testing suite for analog circuit synthesis," *arXiv preprint arXiv:2409.08534*, 2024.